RELATÓRIO TÉCNICO – PARTE PRÁTICA

Pedidos Veloz: implementação fim a fim – Docker Compose, Kubernetes, CI/CD, observabilidade e IaC

Disciplina: Cloud DevOps – Orchestrating Containers and Micro Services | Aluno(a): [SEU NOME] | Outubro/2026

Repositório: https://github.com/BonAnni92/pedidos-veloz **|** **Vídeo:** [LINK DO YOUTUBE]

1. Visão geral da solução

O repositório entrega o MVP proposto na Parte Teórica, cobrindo do ambiente do desenvolvedor até a produção. A aplicação tem quatro microsserviços em Python 3.12/FastAPI e um PostgreSQL 16:

- **API Gateway**: ponto único de entrada, roteamento e validação na borda.
- **Pedidos**: orquestra o fluxo de compra com compensação.
- **Estoque**: catálogo, reserva com bloqueio de linha (SELECT … FOR UPDATE) e baixa.
- **Pagamentos**: simula o provedor externo, com latência, recusa e falha configuráveis.

Um código transversal comum (`libs/veloz_common`) padroniza logs JSON, métricas, tracing e health checks em todos os serviços. Cada serviço continua independente no domínio e tem banco próprio: `pedidos` e `estoque`.

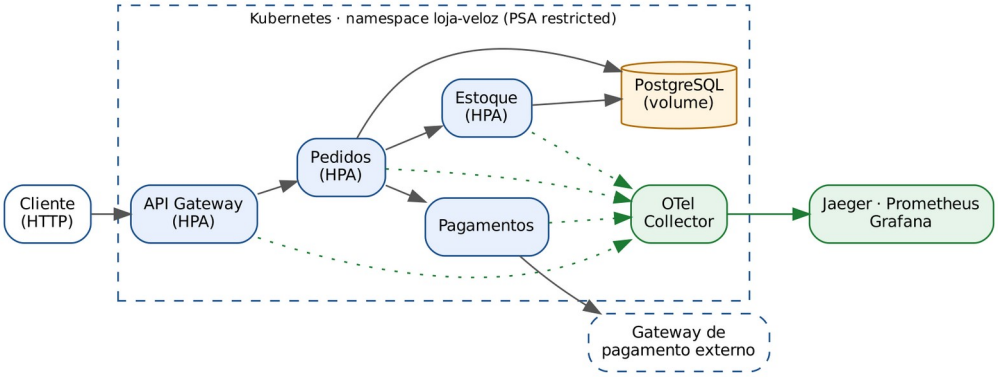


Figura 1 – Arquitetura implementada (namespace loja-veloz no Kubernetes).

Um pedido segue cinco passos: consulta de preço, gravação como PENDENTE, reserva no Estoque, cobrança no Pagamentos e, por fim, confirmação ou cancelamento da reserva. O resultado final é **CONFIRMADO**, **RECUSADO** ou **FALHOU**. Se um serviço cai, o fluxo não deixa estoque “preso”: a reserva é cancelada (padrão saga orquestrada).

2. Ambiente local com Docker Compose

O arquivo `docker-compose.yml` sobe nove contêineres com **um único comando**. São eles: os quatro serviços, o PostgreSQL, o OpenTelemetry Collector, o Jaeger, o Prometheus e o Grafana.

```
docker compose up -d --build        # API em http://localhost:8080/docs
```

- **Redes:** `frontend` (somente o que é acessado do host: gateway, Grafana, Jaeger, Prometheus) e `backend` com `internal: true` (rede isolada, sem saída para a internet).
- **Volumes:** `pgdata` (persistência do banco) e `promdata`; o `db/init.sql` cria um banco por serviço.
- **Variáveis de ambiente:** valores padrão no próprio Compose (`${VAR:-padrão}`), sobrescritos por um `.env` que não é versionado (há o modelo `.env.example`). Âncoras YAML evitam repetir a configuração comum.
- **Ordem e saúde:** `depends_on` com `condition: service_healthy`, healthchecks em todos os serviços e limites de CPU/memória.

Quem não tem Docker instalado pode usar o `.devcontainer`: ele abre o projeto pronto no GitHub Codespaces. O `README.md` traz as instruções completas de execução.

3. Conteinerização e versionamento

Os Dockerfiles usam build multi-stage:

- **builder**: instala as dependências, com versões fixas, em um virtualenv.
- **runtime**: parte de `python:3.12-slim` e recebe só o virtualenv e o código. Compiladores e cache do pip ficam de fora.

As boas práticas de segurança aplicadas são:

- usuário não-root com UID fixo 10001;
- arquivos do código pertencentes ao root, portanto somente leitura para o processo;
- `.dockerignore` que impede o envio de `.env`, testes e `.git` ao build;
- labels OCI com versão e commit;
- `CMD` em forma exec, para que o SIGTERM do Kubernetes chegue ao uvicorn e o desligamento seja gracioso.

O hadolint valida os Dockerfiles no pipeline. As imagens são publicadas no GitHub Container Registry (`ghcr.io/bonanni92/pedidos-veloz-<serviço>`) com as tags abaixo.

Tag	Gerada quando	Uso
sha-<commit> (ex.: sha-3f2a9c1)	todo push na main e tags	Imutável e rastreável até o commit; é a tag usada no deploy e no rollback.
1.2.3 e 1.2 (semver)	tag Git v1.2.3	Releases versionadas semanticamente (arquivo VERSION).
latest	push na branch main	Apenas conveniência; nunca referenciada em produção.

4. Kubernetes – produção mínima

Os manifests estão em `k8s/` e usam Kustomize. A `base` é comum a todos os ambientes, e os overlays `local` (Rancher Desktop/kind) e `prod` mudam apenas o necessário: tag da imagem, Secret, réplicas mínimas e Ingress com TLS. A stack de observabilidade fica em um namespace próprio.

Recurso	Implementação e justificativa
Deployment + Service	Um por serviço; Services ClusterIP (internos). Só o gateway é exposto, via Ingress no overlay de produção.
ConfigMap / Secret	veloz-config com URLs, nível de log e parâmetros. Credenciais no Secret veloz-db-credentials: gerado pelo Kustomize no ambiente local (o hash no nome força o rollout quando a senha muda) e criado pelo pipeline a partir dos GitHub Secrets em produção. Nenhuma senha real fica no Git.
Probes	startupProbe (tolera a espera pelo banco); readinessProbe em /health/ready (testa o banco); livenessProbe em /health/live (sem dependências, para evitar reinícios em cascata).
Pod Security Admission	Namespace com enforce/audit/warn = restricted: o API server rejeita Pods que rodem como root ou permitam escalonamento de privilégio. Os Pods declaram runAsNonRoot, seccomp RuntimeDefault, readOnlyRootFilesystem, drop ALL e automountServiceAccountToken: false. É o mecanismo nativo que substituiu a PodSecurityPolicy.
NetworkPolicy	Default deny de entrada; liberados apenas gateway→pedidos/estoque, pedidos→estoque/pagamentos, pedidos/estoque→postgres e a coleta de métricas pelo Prometheus.
Resiliência	requests/limits, PodDisruptionBudget (mínimo 1), topologySpreadConstraints entre nós, preStop + terminationGracePeriodSeconds.
PostgreSQL	StatefulSet com PVC, também não-root (UID 70). Em produção, a recomendação é um banco gerenciado (RDS) provisionado pelo Terraform.

5. Pipeline CI/CD (GitHub Actions)

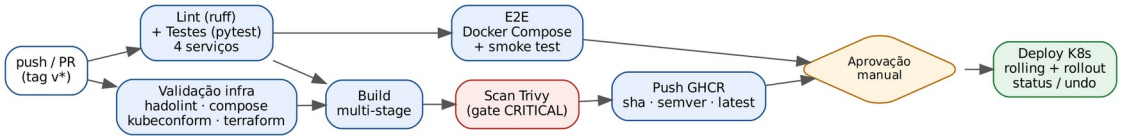


Figura 2 – Estágios do pipeline (.github/workflows/ci-cd.yml).

As etapas do pipeline são:

- **Lint e testes:** ruff e pytest com cobertura, em matriz por serviço, com relatórios JUnit publicados como artifacts.
- **Validação de infraestrutura:** hadolint, `docker compose config`, kubeconform nos três conjuntos de manifests, além de `terraform fmt` e `terraform validate`.
- **Teste ponta a ponta:** sobe o Compose com `--wait` e executa o smoke test pelo gateway.

- **Build, scan e publicação:** cache de camadas (`type=gha`); o Trivy reprova a imagem se houver vulnerabilidade CRITICAL com correção; o push ao GHCR acontece só fora de pull requests.
- **Deploy (tags v*):** passa pelo environment `production`, que exige aprovação manual. O job cria o Secret, fixa a tag `sha-<commit>`, aplica os manifests e acompanha `rollout status`. Em caso de falha, faz `rollout undo` automático.

Secrets: o `GITHUB_TOKEN` (automático, com permissão mínima `packages: write`) publica as imagens. `KUBE_CONFIG`, `DB_USER` e `DB_PASSWORD` ficam no environment, nunca no código ou nos logs.

6. Observabilidade, deploy e escala

A observabilidade usa três sinais:

- **Métricas:** cada serviço expõe `/metrics` (req/s, latência por histograma, códigos de status) e métricas de negócio (`pedidos_criados_total{status}`, `pagamentos_total{resultado}`). O Prometheus coleta por descoberta de Pods (anotações), e o Grafana já vem provisionado com o painel "Pedidos Veloz – Visão Geral": req/s, p95, taxa de 5xx, pedidos por status e instâncias.
- **Logs:** JSON no stdout (12-Factor), com service, trace_id e span_id.
- **Tracing distribuído (instrumentado):** OpenTelemetry com auto-instrumentação de FastAPI, httpx e SQLAlchemy. O contexto W3C traceparent é propagado entre os serviços e exportado via OTLP ao Collector, que encaminha ao Jaeger. Em produção, o Collector amostra 20% dos traces. Um único trace mostra gateway → pedidos → estoque → pagamentos, inclusive as queries SQL, o que localiza exatamente onde houve lentidão ou erro.

Para provocar falhas e vê-las nos traces, basta ajustar `PAGAMENTO_TAXA_FALHA`. O service mesh (Istio, com mTLS e canary) ficou como evolução, pelo custo operacional dos sidecars no prazo de 4 semanas.

Estratégia de deploy e escalabilidade:

- **Deploy:** Rolling Update com `maxSurge: 1` e `maxUnavailable: 0`. Cada Pod novo só substitui um antigo depois de passar na readiness probe, então a capacidade nunca cai. O `revisionHistoryLimit` permite rollback imediato. Foi escolhido por ser nativo, sem infraestrutura duplicada (Blue-Green) nem mesh (Canary).
- **Escala horizontal (HPA):** por CPU (alvo de 70%) no gateway e no pedidos (2–10 réplicas; mínimo 3 em produção) e no estoque (2–6). Escala para cima rápido (+4 Pods a cada 15 s) e para baixo com janela de 5 min, evitando oscilação.
- **Pagamentos:** réplicas fixas, porque o gargalo é o limite do provedor externo, não CPU.
- **VPA:** em modo recomendação (`updateMode: Off`), para não conflitar com o HPA na mesma métrica.

7. Infraestrutura como código (Terraform)

A pasta `terraform/` traz um esqueleto modular e validado:

- `modules/cluster`: VPC em múltiplas AZs e Amazon EKS com node group gerenciado, usando os módulos terraform-aws-modules em versões fixas.
- `modules/app-namespace`: namespace com PSA restricted, ResourceQuota e LimitRange.
- `envs/dev` e `envs/prod`: compõem os módulos com o mesmo código, mudando só o `terraform.tfvars`. Dev usa 2 AZs e 1 NAT; prod usa 3 AZs, NAT por AZ e mais nós.

O state é remoto em S3, com lock, para que o time e o pipeline compartilhem o mesmo estado. Com isso, a infraestrutura pode ser recriada de forma idêntica e revisada por pull request.

8. Evidências de validação

Verificação	Resultado
Testes automatizados (pytest)	33 testes aprovados: estoque 10, pedidos 9, gateway 8, pagamentos 6. Cobrem fluxo feliz, estoque insuficiente, pagamento recusado, provedor fora do ar com compensação, idempotência, validação e métricas.
Execução integrada dos 4 serviços	Smoke test aprovado; carga de 8 s com 6 clientes: 1.482 requisições (~185 req/s), 0% de erros, p95 ≈ 104 ms.
Lint	ruff, hadolint e actionlint sem apontamentos.
Manifests Kubernetes	kubeconform (strict, K8s 1.31): 68 recursos válidos (overlays local e prod e observabilidade).
Compose e Terraform	docker compose config válido; terraform fmt e validate aprovados em dev e prod.

Limitações e próximos passos: migrações de schema com Alembic executadas por Job antes do deploy; mensageria (evento PedidoCriado) com padrão outbox; Canary com Istio e rollback por métrica; External Secrets/Vault; alertas por SLO (p95 e taxa de erro).

Referências

DOCKER. Docker Docs. https://docs.docker.com/ · KUBERNETES. Documentation. https://kubernetes.io/docs/

GITHUB. GitHub Actions. https://docs.github.com/actions · HASHICORP. Terraform. https://developer.hashicorp.com/terraform

OPENTELEMETRY. Documentation. https://opentelemetry.io/docs/ · WIGGINS, A. The Twelve-Factor App. https://12factor.net/

GOOGLE CLOUD. Online Boutique. https://github.com/GoogleCloudPlatform/microservices-demo. Acesso em: out. 2026.